# CAPSTONE PROJECT REPORT

## MINI-COURIER PLANNER


### Submitted by

Name: Karnav Tomar

University: K.R. Mangalam University

Roll Number: 2501730127

Programme: B.Tech CSE (AI/ML)

Semester: 3

Academic Year: 2026


Submitted as part of the Capstone Assignment: Mini-Courier Planner.

## 1. Problem Statement

Build a prototype that accepts a list of parcels and delivery points, generates an ordered delivery route, and creates an optimized packing plan. The application integrates sorting, dynamic programming, graph algorithms, and approximation techniques.

## 2. Objectives

The project aims to prioritize parcels by deadline and priority, select a valuable set of parcels within the van capacity, find shortest paths in a weighted delivery graph, construct an approximate delivery tour, and measure execution time.

## 3. System Design

The system is divided into Python modules for data loading, sorting, knapsack optimization, shortest paths, route approximation, visualization, benchmarking, and application integration. Parcel and road data are stored in CSV files.

## 4. Parcel Prioritization

The sorting module orders parcels by earliest delivery deadline. For equal deadlines, HIGH priority is placed before MEDIUM and LOW. Parcel ID is used as a deterministic tie-breaker. Python's built-in Timsort has O(n log n) worst-case time complexity.

## 5. Package Selection — 0/1 Knapsack

The dynamic programming module selects parcels to maximize total parcel value without exceeding the van capacity. Each parcel can be selected at most once. The algorithm has O(nW) time and space complexity, where n is the number of parcels and W is the integer capacity.

## 6. Shortest Path — Dijkstra

The delivery network is represented as a weighted undirected adjacency graph. Dijkstra's algorithm uses a min-priority queue to calculate shortest paths with non-negative edge weights. With a binary heap, its complexity is O((V+E) log V).

## 7. Route Planning — Nearest-Neighbour TSP

Starting from DEPOT, the algorithm repeatedly visits the nearest unvisited destination using shortest-path distances, then returns to the depot. This is a greedy TSP approximation and does not guarantee the globally shortest possible tour.

## 8. Sample Data and Configuration

The supplied sample dataset contains eight parcels and delivery destinations A through H. The default van capacity is 20 kg. The routes CSV stores undirected road connections and distances in kilometres.

## 9. Testing and Performance

The application can be run with `python -m src.main`. The benchmark can be run with `python -m src.benchmark`, which tests input sizes 10, 30, 60, and 100 and writes measurements to outputs/benchmark_results.csv. Runtime values vary with hardware and system load; the submitted computer's measured results should be used for final observations.

## 10. Sample Application Result

The included sample run selected parcels with a combined weight of 20 kg out of the 20 kg capacity. The sample nearest-neighbour tour was DEPOT → A → B → D → F → E → DEPOT, with a total distance of 29.0 km. The exact output is saved in outputs/console_output.txt.

## 11. Complexity Summary

Sorting: O(n log n). 0/1 Knapsack: O(nW). Dijkstra with a binary heap: O((V+E) log V). The nearest-neighbour tour repeatedly runs shortest-path searches for the remaining destinations, so its cost depends on the number of delivery

points and graph size.

## 12. Conclusion

The Mini-Courier Planner demonstrates how sorting, dynamic programming, shortest-path graph algorithms, and a greedy approximation can be combined into a modular Python application. It provides an educational prototype for parcel selection and route planning.

## 13. Limitations and Future Scope

The current prototype uses sample data, one depot, one vehicle, an undirected graph, and non-negative distances. Future work could add multiple vehicles, time windows, traffic-aware travel times, one-way roads, live map data, and route-improvement methods such as 2-opt.

## 14. Project Files

| Folder/File | Purpose |
|---|---|
| src/ | Python modules implementing sorting, knapsack, Dijkstra, TSP, benchmarking and visualization. |
| datasets/ | Sample parcel and weighted route CSV files. |
| images/ | Generated route and performance charts. |
| outputs/ | Program output, JSON result and benchmark CSV. |
| mini_courier_planner.ipynb | Notebook entry point for running the project. |
| README.md | Project overview, setup and execution instructions. |

Before final submission, ensure screenshots captured from the student's own VS Code execution are included in the images folder, and confirm that the instructor's required declaration or cover-page format is followed.